KHTML to WebKit
This is still what drives Konqueror, KDE's multi-purpose web and file browser

Vote now
Vote for the young Indian innovators who are changing the world through their contributions. Head to www.thinkdigit.com/yii

What drives the browser?

What is this browser engine thingy, really, and why must we care?

Nimish Chandiramani
readersletters@thinkdigit.com

The browser wars aren't just about the feature-rich against the featureless, nor about open source software butting heads with Big Evil Corporation #22935. It's really the underlying technology – the browser engines – at war here: the way these browsers take the mess that is HTML code, and turn it into something pretty for us to see.

Why, though, does the browser engine suddenly matter? We're happy to compare programs on their features, because it's really the features we're using. Right?

But there's more to the browser than the features. With a great browser engine comes the ability to take that engine everywhere: from Windows to Linux to the Mac to the mobile phone to the game console to the Magic Intertube-surfing Shoe. A classic example is WebKit, which started out as the wind beneath Safari's wings, but now powers Google Chrome, the Android web browser, the web browser for Symbian, Safari Mobile, and such obscure but well-loved browsers as OmniWeb and iCab.

The simplest engine

Let us consider a hypothetical browser engine whose only purpose is to take HTML and convert it into a displayed web page – none of these fancy JavaScript or CSS designs for this one. As it receives HTML code, it finds an opening tag – `<b>`, for example – and recognises that it needs to start spewing out bold text until it reaches the closing `</b>` tag.

That, then, is what browser engines are supposed to do – whatever the HTML tag tells them to. If you're old enough to remember WordStar, it's the same concept – tell the program to start bolding text here, and stop there.

But if you do want that fancy JavaScript and CSS stuff, this approach doesn't work, because you want your document all ready and rendered first, so that you can use JavaScript programming to add some interactivity. And so came the Document Object Model.

Treat them like objects

Once JavaScript was born, the simple read-and-spew approach wouldn't do at all. JavaScript needs to take web pages and manipulate them, and the only way to do that is to turn an HTML document into a collection of objects. The document itself, in fact, is an object.

Now, instead of HTML turning into a stream of text, the browser turns an HTML page into a Document object, with all HTML elements within becoming child objects – paragraphs, forms, links and so on – creating a hierarchical tree of objects. This object-like representation is called the *document object model* (DOM), and it opens up a host of possibilities.

In object-oriented programming (OOP), objects are wonderful things. They are balls of programmatic clay, which can be moulded and manipulated as the programmer sees fit. If, for example, browser developers decide that all forms should have a green halo around them, they can do that – the HTML code itself doesn't need to change. And now, pages can become less boring, too.

By implementing a DOM, browser developers gave web designers their greatest

Imaging Chaitanya Surpur